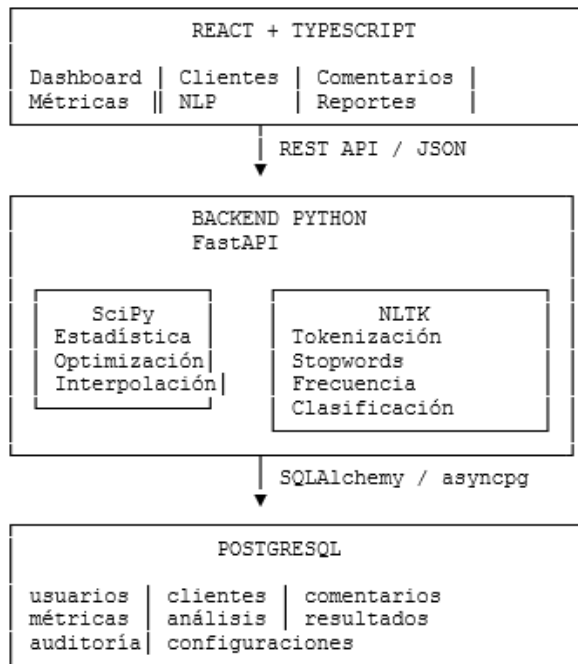


1. Arquitectura propuesta

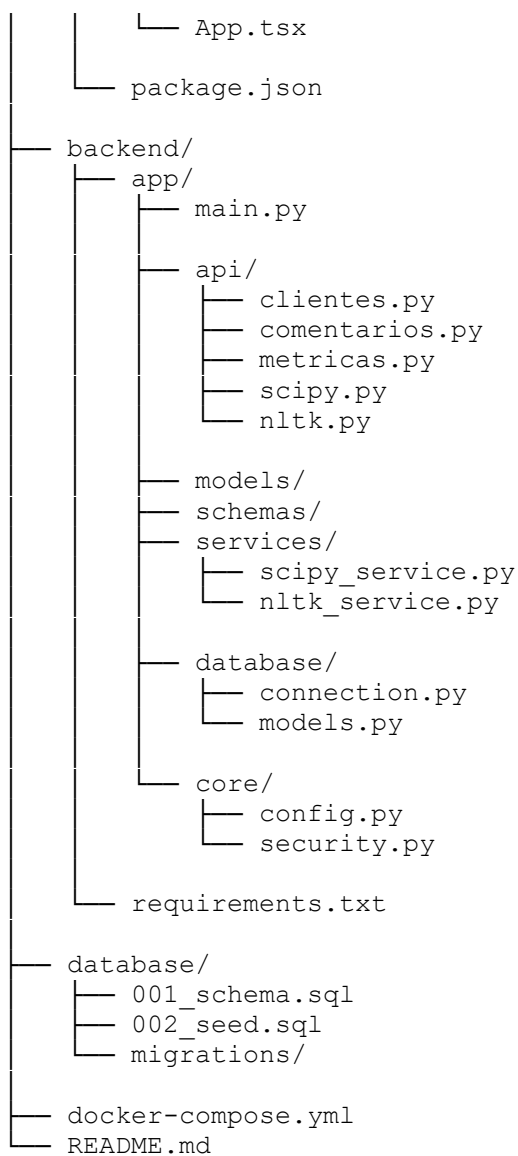


La idea importante es que **SciPy y NLTK estén en Python**, no directamente en React. React se encarga de la interfaz y Python del procesamiento científico/NLP.

2. Estructura del proyecto

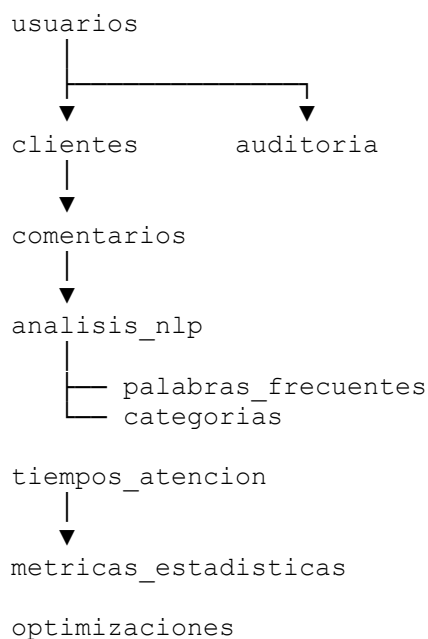
Te recomiendo esta estructura:

```
empresa-inteligente/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   │   ├── ui/  
│   │   │   ├── dashboard/  
│   │   │   ├── comentarios/  
│   │   │   └── metricas/  
│   │   ├── pages/  
│   │   │   ├── Dashboard.tsx  
│   │   │   ├── Clientes.tsx  
│   │   │   ├── Comentarios.tsx  
│   │   │   ├── AnalisisNLP.tsx  
│   │   │   ├── Metricas.tsx  
│   │   │   ├── Optimizacion.tsx  
│   │   │   └── Reportes.tsx  
│   │   ├── services/  
│   │   │   ├── api.ts  
│   │   │   ├── comentarios.ts  
│   │   │   ├── scipy.ts  
│   │   │   └── nltk.ts  
│   │   ├── hooks/  
│   │   ├── types/  
│   │   ├── layouts/  
│   │   └── routes/
```



3. Base de datos PostgreSQL

Para una primera versión empresarial propongo estas tablas:



4. Tabla de usuarios

```
CREATE TABLE usuarios (  
    id BIGSERIAL PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    email VARCHAR(200) UNIQUE NOT NULL,  
    password_hash TEXT NOT NULL,  
    rol VARCHAR(30) NOT NULL DEFAULT 'usuario',  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Roles:

ADMIN
ANALISTA
SUPERVISOR
USUARIO

5. Clientes

```
CREATE TABLE clientes (  
    id BIGSERIAL PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    email VARCHAR(200),  
    telefono VARCHAR(50),  
    empresa VARCHAR(200),  
    activo BOOLEAN DEFAULT TRUE,  
    created_at TIMESTAMPTZ DEFAULT NOW(),  
    updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

6. Comentarios de clientes

Esta será una de las tablas principales para NLTK.

```
CREATE TABLE comentarios (  
    id BIGSERIAL PRIMARY KEY,  
  
    cliente_id BIGINT REFERENCES clientes(id)  
        ON DELETE SET NULL,  
  
    contenido TEXT NOT NULL,  
  
    canal VARCHAR(30) DEFAULT 'web',  
  
    estado VARCHAR(30) DEFAULT 'pendiente',  
  
    categoria VARCHAR(50),  
  
    fecha TIMESTAMPTZ DEFAULT NOW(),  
  
    procesado BOOLEAN DEFAULT FALSE  
);
```

Ejemplo:

"El servicio fue rápido y la atención excelente"

NLTK podrá procesar ese comentario.

7. Análisis NLP

Cada comentario puede tener un análisis asociado.

```
CREATE TABLE analisis_nlp (  
  id BIGSERIAL PRIMARY KEY,  
  
  comentario_id BIGINT NOT NULL  
    REFERENCES comentarios(id)  
    ON DELETE CASCADE,  
  
  idioma VARCHAR(20) DEFAULT 'es',  
  
  cantidad_palabras INTEGER DEFAULT 0,  
  
  palabras_limpias JSONB,  
  
  palabras_frecuentes JSONB,  
  
  categoria_detectada VARCHAR(100),  
  
  confianza NUMERIC(5,4),  
  
  fecha_analisis TIMESTAMPTZ DEFAULT NOW()  
);
```

Ejemplo:

```
{  
  "palabras_frecuentes": [  
    {  
      "palabra": "servicio",  
      "frecuencia": 5  
    },  
    {  
      "palabra": "atención",  
      "frecuencia": 3  
    }  
  ]  
}
```

8. Categorías de comentarios

Podemos crear categorías configurables.

```
CREATE TABLE categorias (  
  id BIGSERIAL PRIMARY KEY,  
  
  nombre VARCHAR(100) NOT NULL UNIQUE,  
  
  descripcion TEXT,  
  
  activo BOOLEAN DEFAULT TRUE,  
  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Inicialmente:

VENTAS
SOPORTE
RECLAMO
CONSULTA
FELICITACION
OTROS

9. Tiempos de atención

Esta tabla alimentará SciPy.

```
CREATE TABLE tiempos_atencion (  
    id BIGSERIAL PRIMARY KEY,  
  
    cliente_id BIGINT REFERENCES clientes(id)  
        ON DELETE SET NULL,  
  
    comentario_id BIGINT REFERENCES comentarios(id)  
        ON DELETE SET NULL,  
  
    tiempo_minutos NUMERIC(10,2) NOT NULL,  
  
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,  
  
    operador VARCHAR(150),  
  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

Ejemplo:

Cliente	Tiempo

Cliente 01	12
Cliente 02	15
Cliente 03	18
Cliente 04	20
Cliente 05	11

10. Métricas estadísticas

Aquí almacenaremos los resultados calculados con SciPy.

```
CREATE TABLE metricas_estadisticas (  
    id BIGSERIAL PRIMARY KEY,  
  
    fecha_inicio DATE NOT NULL,  
  
    fecha_fin DATE NOT NULL,  
  
    cantidad_registros INTEGER NOT NULL,  
  
    media NUMERIC(12,4),  
  
    mediana NUMERIC(12,4),  
  
    desviacion_estandar NUMERIC(12,4),  
  
    minimo NUMERIC(12,4),  
  
    maximo NUMERIC(12,4),
```

```

percentil_25 NUMERIC(12,4),

percentil_75 NUMERIC(12,4),

created_at TIMESTAMPTZ DEFAULT NOW()
);

```

Esto permitirá mostrar:

TIEMPO PROMEDIO 17.2 min

DESVIACIÓN ESTÁNDAR 4.3

TIEMPO MÁXIMO 25 min

11. Optimización con SciPy

Para guardar escenarios de optimización:

```

CREATE TABLE optimizaciones (
  id BIGSERIAL PRIMARY KEY,

  nombre VARCHAR(150) NOT NULL,

  descripcion TEXT,

  parametros_entrada JSONB NOT NULL,

  resultado JSONB,

  costo_inicial NUMERIC(14,4),

  costo_optimizado NUMERIC(14,4),

  estado VARCHAR(30) DEFAULT 'pendiente',

  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

Por ejemplo:

```

{
  "recurso_a": 3,
  "recurso_b": 5
}

```

Resultado:

```
{
  "recurso_a": 2.8,
  "recurso_b": 4.1,
  "costo": 742.50
}
```

12. Auditoría

En una aplicación empresarial es importante registrar acciones.

```
CREATE TABLE auditoria (
  id BIGSERIAL PRIMARY KEY,

  usuario_id BIGINT REFERENCES usuarios(id)
    ON DELETE SET NULL,

  accion VARCHAR(100) NOT NULL,

  tabla VARCHAR(100),

  registro_id BIGINT,

  detalles JSONB,

  ip VARCHAR(45),

  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

13. Funcionalidades del sistema

La aplicación podría tener este menú:

```
DASHBOARD
├── Inicio
├── CLIENTES
│   ├── Lista de clientes
│   ├── Nuevo cliente
│   └── Historial
├── ATENCIÓN
│   ├── Solicitudes
│   ├── Comentarios
│   └── Tiempos de atención
├── INTELIGENCIA NLP
│   ├── Analizar comentario
│   ├── Palabras frecuentes
│   ├── Categorías
│   └── Clasificación
├── SCIENTIFIC DATA
│   ├── Estadísticas
│   ├── Interpolación
│   └── Optimización
└── REPORTES
    ├── Atención
    ├── NLP
    └── Estadísticas
```

14. Dashboard principal

El dashboard debería ser la parte más importante.

CENTRO INTELIGENTE			
CLIENTES 245	COMENTARIOS 1,248	PROMEDIO 16.4 min	PROCESADOS 94%

TIEMPOS DE ATENCIÓN	CATEGORÍAS NLP
 Gráfico	Soporte 42%
	Ventas 27%
	Reclamos 18%

PALABRAS MÁS FRECUENTES				
servicio	atención	rápido	producto	soporte

15. API del backend

FastAPI podría tener:

Clientes

```
GET    /api/clientes
GET    /api/clientes/{id}
POST   /api/clientes
PUT    /api/clientes/{id}
DELETE /api/clientes/{id}
```

Comentarios

```
GET    /api/comentarios
POST   /api/comentarios
GET    /api/comentarios/{id}
DELETE /api/comentarios/{id}
```

NLTK

```
POST /api/nltk/analizar
POST /api/nltk/palabras-frecuentes
POST /api/nltk/clasificar
```

Por ejemplo:

```
POST /api/nltk/analizar
```

```
{
```



```
    "texto": "El servicio fue excelente y rápido"
  }
```

Respuesta:

```
{
  "idioma": "es",
  "cantidad_palabras": 7,
  "tokens": [
    "servicio",
    "excelente",
    "rápido"
  ],
  "palabras_frecuentes": [
    {
      "palabra": "servicio",
      "frecuencia": 1
    }
  ],
  "categoria": "FELICITACION"
}
```

16. API SciPy

```
GET /api/scipy/estadisticas
POST /api/scipy/estadisticas
POST /api/scipy/optimizacion
POST /api/scipy/interpolacion
```

Ejemplo:

```
POST /api/scipy/estadisticas
{
  "valores": [12,15,18,20,11,25,19,17,14,21]
}
```

Respuesta:

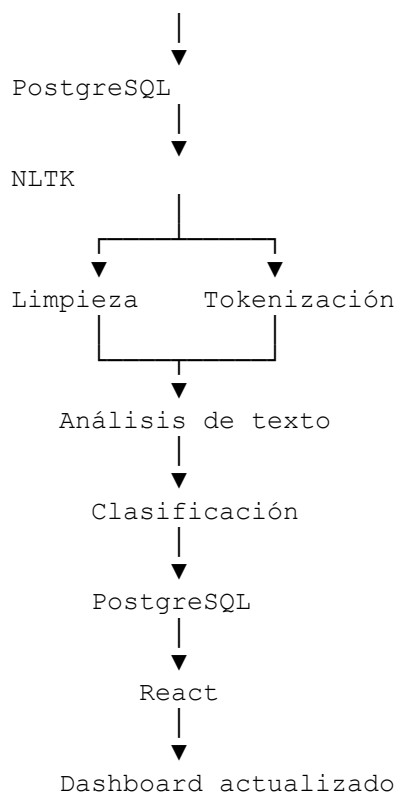
```
{
  "cantidad": 10,
  "media": 17.2,
  "mediana": 17.5,
  "desviacion_estandar": 4.44,
  "minimo": 11,
  "maximo": 25
}
```

React solamente consume esta información.

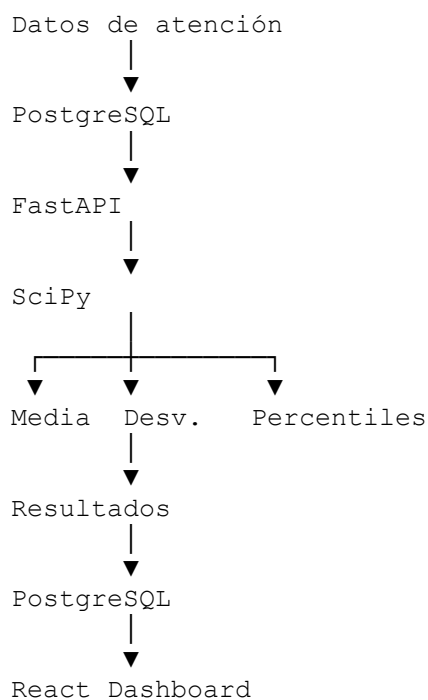
17. Flujo de un comentario

La funcionalidad sería:

```
Cliente escribe comentario
    |
    ▼
React
    |
    ▼
POST /api/comentarios
```



18. Flujo de SciPy



19. Tecnologías recomendadas

Frontend

React
TypeScript
Vite
Tailwind CSS
React Router / TanStack Router
Axios o fetch

Recharts / Chart.js
Lucide React
React Hook Form
Zod

Backend

Python
FastAPI
SQLAlchemy
Pydantic
SciPy
NumPy
NLTK
Uvicorn
Alembic

Base de datos

PostgreSQL

Producción

Docker
Nginx
HTTPS
Variables de entorno
Logs
Backups PostgreSQL

20. Fases de desarrollo

Yo lo desarrollaría en este orden:

FASE 1 — Base

- PostgreSQL.
- Migraciones.
- Usuarios.
- Clientes.
- Configuración.

FASE 2 — Atención

- Registro de comentarios.
- Registro de tiempos.
- Historial.
- Estados.

FASE 3 — NLTK

- Tokenización.
- Stopwords.
- Normalización.
- Frecuencia.
- Clasificación.
- Dashboard NLP.

FASE 4 — SciPy

- Estadísticas.
- Desviación estándar.
- Percentiles.
- Interpolación.
- Optimización.

FASE 5 — Dashboard

- KPIs.
- Gráficos.
- Filtros.
- Alertas.
- Indicadores empresariales.

FASE 6 — Producción

- Autenticación.
- Roles.
- Auditoría.
- Validaciones.
- Manejo de errores.
- Docker.
- Backups.
- Seguridad.

Recomendación importante

No haría que React ejecute directamente SciPy o NLTK. La arquitectura correcta sería:

React + TypeScript → FastAPI → SciPy/NLTK → PostgreSQL

Así tendrás una aplicación que posteriormente puede crecer hacia **IA, análisis predictivo, clasificación avanzada de clientes y automatización empresarial** sin tener que rehacer toda la estructura.